

Unit V

5

IoT Physical Servers and Cloud Offerings

5.1 : Introduction to Cloud Storage Model and Communication APIs

Q.1 Define cloud computing ? Explain components of cloud computing.

Ans. : • **NIST definition of cloud :** Cloud computing is a pay-per-use model for enabling available, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, services) that can be rapidly provisioned and released with minimal management effort or service-provider interaction.

Components of Cloud Computing

• Cloud computing solutions are made up of several elements. Fig. Q.1.1 shows cloud components.

1. **Clients :** Mobile, terminals or regular computers.
2. **Benefits :** Lower hardware costs, lower IT costs, security, data security, less power consumption, ease of repair or replacement, less noise.
3. **Data centers :** Collection of servers where the application to subscribe is housed. It could be a large room in the basement of your building or a room full of servers on the other side of the world.
4. **Virtualizing servers :** Software can be installed allowing multiple instances of virtual servers to be used and a dozen virtual servers can run on one physical server.
5. **Distributed servers :** Servers don't all have to be housed in the same location. It can be in geographically disparate locations. If something were to happen at one site, causing a failure, the service

(5 - 1)

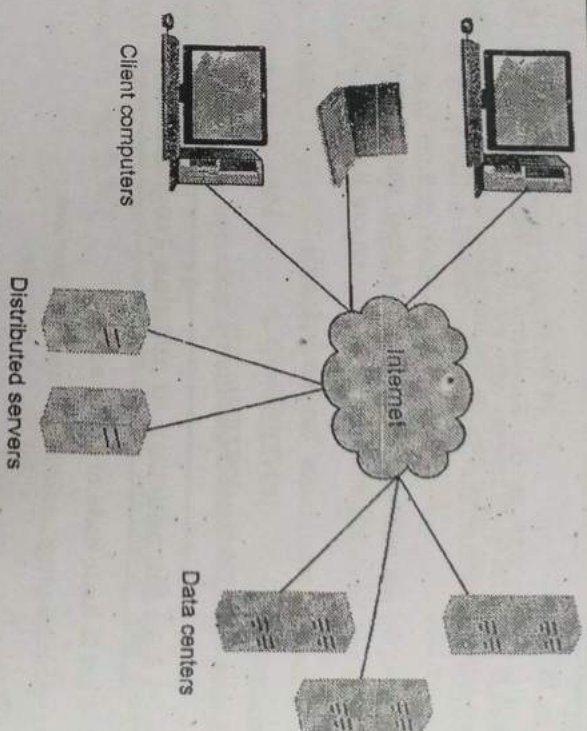


Fig. Q.1.1 Cloud components

would still be accessed through another site. If the cloud needs more hardware, they can add them at another site.

Q.2 Explain characteristics of cloud computing.

Ans. : Characteristics of Cloud Computing

1. **On-demand self-service :** A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed without requiring human interaction with each service's provider.
2. **Ubiquitous network access :** Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms.
3. **Location-independent resource pooling :** The provider's computing resources are pooled to serve all consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.
4. **Rapid elasticity :** Capabilities can be rapidly and elastically provisioned to quickly scale up, and rapidly released to quickly scale down.

5. **Pay per use** : Capabilities are charged using a metered, fee-for-service, or advertising-based billing model to promote optimization of resource use.

Q.3 What is software as a service ? Explain characteristics of SaaS.

Ans. : • Software as a Service (SaaS) is a way of delivering applications over the Internet as a service. Instead of installing and maintaining software, user simply access it via the Internet, freeing yourself from complex software and hardware management.

- SaaS applications are sometimes called web-based software, on-demand software, or hosted software.
- The public cloud provider manages all the hardware and traditional software, including middleware, application software and security. So SaaS customers can dramatically lower costs.
- In this model, the user, client or consumer runs an application from a cloud infrastructure. Through an interface such as a web browser, the client or user may access this application from a variety of devices.
- The complete application is offered as on demand service. This saves the client from having to invest in any software licenses or servers up front and can save the provider money since they are maintaining and providing only a single application.

Characteristics of SaaS :

1. Software applications or services are stored remotely.
2. A user can then access these services or software applications via the Internet.
3. In most cases, a user does not have to install anything onto their host machine, all they require is a web browser to access these services and in some cases, a browser may require additional plug-in/add-on for certain services.
4. Network-based management and access to commercially available software from central locations rather than at each customer's site, enabling customers to access applications remotely via the Internet.
5. Application delivery from a one-to-many model, as opposed to a traditional one-to-one model.

Q.4 What is PaaS ? Explain PaaS characteristic and benefit.

Ans. : • Platform as a Service is another application delivery model and also known as cloud-ware. Supplies all the resources required to build applications and services completely from the Internet, without having to download or install software.

- Services include : Application design, development, testing, deployment, and hosting, team collaboration, web service integration, database integration, security, scalability, storage, state management, and versioning.
- PaaS is closely related to SaaS but delivers a platform from which to work rather than an application to work with.
- This model involves software encapsulated and offered as a service, from which higher levels of service may then be built. The user, customer, or client in this model is the one building applications which then run on the provider's infrastructure.
- This in turn provides customers and clients with the capability to deploy applications onto the cloud infrastructure using programming tools and languages, which the provider supports.
- The customer still does not manage the framework, network, servers or operating system, but has control over deployed applications and sometimes over the hosting environment itself.
- Some examples of Platform as a Service include Google's App Engine or Force.com
- PaaS consists of following components :
 1. Browser based development studio
 2. Pay contrary to billing
 3. Management and supervising tools
 4. Seamless deployment to host run time environment.

• Characteristics of Paas :

1. It support multi-tenant architecture.
2. It support for development of group collaboration.
3. Paas systems can be deployed as public cloud services or as private cloud services.
4. Provision of runtime environments. Typically each runtime environment supports either one or a small set of programming languages and frameworks.
5. Support for custom applications. Support for the development, deployment and operation of custom applications.
6. Preconfigured capabilities. Many Paas systems are characterized by capabilities that are preconfigured by the provider, with a minimum of configuration available to developers and customer operations staff.
7. Support for porting existing applications. While many Paas systems are primarily designed to support "born on the cloud" applications.
8. Security is an important characteristic in Paas. It needs to provide authentication and authorization to differentiate the access rights of different users.

Benefits of Paas :

1. Scalability including rapid allocation and deallocation of resources with a pay-as-you-use model.
2. Reduced capital expenditure
3. Reduced lead times with on-demand availability of resources
4. Self-service with reduced administration costs
5. Reduced skill requirements
6. Support of team collaboration
7. Ability to add new users quickly.

Q.5 Define Iaas. Explain advantages of Iaas.

Ans. : • Infrastructure as a service hosts infrastructure on the public cloud and private cloud instead of in a traditional on-premises data center. The infrastructure is delivered to customers on demand while being fully managed by the service provider.

Advantages of Iaas :

1. Elimination of an expensive and staff-intensive data center
2. Ease of hardware scalability
3. Reduced hardware cost
4. On-demand, pay as you go scalability
5. Reduction of IT staff
6. Suitability for ad hoc test environments
7. Allows complete system administration and management
8. Support multiple tenants.

Q.6 What is difference between Iaas, Paas and Saas ?

Ans. :

IaaS	Paas	SaaS
IaaS gives users automated and scalable environments	Paas provides a framework for quickly developing and deploying applications.	SaaS makes applications available through the internet.
Amazon Web Services, for example, offers IaaS through the Elastic Compute Cloud or EC2.	Google Cloud Platform provides another Paas option in App Engine.	SaaS applications such as Gmail, Dropbox, Salesforce, or Netflix.
In IaaS, infrastructure as a service.	In Paas, platform as a service.	In SaaS, software as a service.
Virtual platform on which required operating environment and application deployed	Operating environment included.	Operating environment largely irrelevant, fully functional application provided.
IaaS is a cloud service that provides basic computing infrastructure. Servers, storage, and networking resources. In other words, IaaS is a virtual data center.	Paas refers to cloud platforms that provide runtime environments for developing, testing, and managing applications.	SaaS allows people to use cloud-based web applications.

Internet of Things	Examples of Paas	Examples of SaaS
Major IaaS providers include Amazon Web Services, Microsoft Azure, and Google Compute Engine.	Examples of Paas services are Heroku and Google App Engine.	Examples of SaaS services are Gmail and Hotmail.
IaaS services are available on a pay-for-what-you-use model.	Paas solutions are available with a pay-as-you-go pricing model.	SaaS services are usually available with a pay-as-you-go pricing model.
Used by IT administrator.	Used by software developers.	Used by end user.

Q.7 What is cloud storage ? Briefly discuss advantages of cloud storage.
 [SPPU : May-19, End Sem, Marks 8]

- Ans. : • Cloud storage is a cloud computing model that stores data on the Internet through a cloud computing provider who manages and operates data storage as a service. It's delivered on demand with just-in-time capacity and costs and eliminates buying and managing your own data storage infrastructure.
- Cloud storage is based on a virtualized storage infrastructure with accessible interfaces, near-instant elasticity and scalability, multi-tenancy and metered resources.
 - Cloud storage includes at least one data server to which a user can connect via the internet. The user sends files to the data server, which forwards the message to multiple servers, manually or in an automated manner, over the internet. The stored data can then be accessed via a web-based interface.
- Cloud storage advantages :**
1. Organizations need only pay for the storage they actually use as it is also possible for companies by utilizing actual virtual storage features like thin provisioning.
 2. Companies do not need to install physical storage devices in their own data centre or offices.

3. Storage maintenance tasks, such as backup, data replication and purchasing additional storage devices are offloaded to the responsibility of a service provider, allowing organizations to focus on their core business.
4. Cloud storage provides users with immediate access to a broad range of resource and applications hosted in the infrastructure of another organization via a web service interface.
5. Cloud storage can be used for copying virtual machine images from the cloud to on-premise locations or to import a virtual machine image from an on-premise location to the cloud image library.

5.2 : WAMP

Q.8 What is WAMP ? List the key components of WAMP. What is autobahn ?
 [SPPU : May-19, End Sem, Marks 10]

- Ans. : • WAMP stands for web application messaging protocol. It is sub-protocol of web-socket and provides publish-subscribe and RPC messaging pattern. WAMP is a routed protocol, with all components connecting to a WAMP router, where the WAMP router performs message routing between the components.
- It support distributed application architecture. Application components are distributed and run on mobile nodes and WAMP provides communication in between them.
 - WAMP does not make any extra standard, but it bases on established Web standards : WebSocket, JSON and URL. The WebSocket standard is the default transport channel where WAMP is binding. With the WebSocket Standard, WAMP is assumed to be given a reliable, ordered and full-duplex message channel.
 - The PubSub messaging pattern defines three roles : Subscriber and publishers, which communicate via a broker. The routed RPC messaging

pattern also defines three roles : Callers and Calleees, which communicate via a Dealer.

• Key components of WAMP :

1. **Transport** : It is communication channel which connects two peers. WebSocket is default transport for WAMP. WAMP also uses other transport protocol which is reliable and bi-directional communication.
2. **Session** : Conversation between two is called as session.
3. **Client** : Clients are peers that can have one or more roles.

Autobahn

• Autobahn is an open-source real-time framework enabling implementations of the WAMP and WebSocket protocols using multiple languages that are supported by Taverdo. These languages include Autobahn Python (based on Python), Autobahn JS (based on JavaScript), Autobahn Cpp (based on C++) and Autobahn Android.

• Each language is considered a subproject with its own sub-community and developers. Autobahn Python is utilized as part of Crossbar.io itself. While these four libraries are supported by Taverdo, other libraries are provided by third-parties including ones based on other languages not supported by Taverdo such as C#, Java and PHP. Autobahn also includes a fully automated testing tool called Autobahn Test suite.

Q.9 Explain WAMP session between client and router.

Ans. : • Fig. Q.9.1 shows WAMP session between client and router.

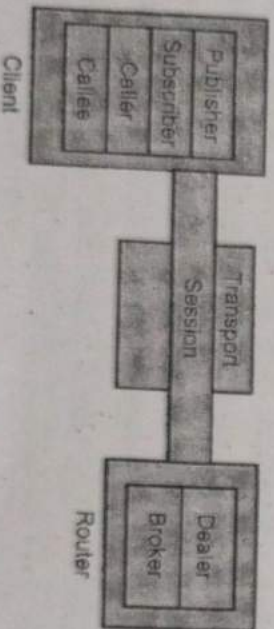


Fig. Q.9.1 WAMP session between client and router

• A WAMP Session connects two Peers, a Client and a Router. Each WAMP peer MUST implement one role and MAY implement more roles. A Client MAY implement any combination of the Roles : Callee, Caller, Publisher and Subscriber.

• Router MAY implement either or both of the Roles : Dealer and Broker.

• WAMP implementations may choose to tie the lifetime of the underlying transport connection for a WAMP connection to that of a WAMP session, i.e. establish a new transport-layer connection as part of each new session establishment. They may equally choose to allow re-use of a transport connection, allowing subsequent WAMP sessions to be established using the same transport connection.

5.3 : Xively Cloud for IoT

Q.10 Write short note on : Xively.

Ans. : • Xively is a system for deploying IoT applications on the cloud. It is offered as PaaS. Xively is basically a data collection, management and distribution infrastructure. It also provides APIs to connect and develop IoT applications.

• Xively's management services enable provisioning and management of devices, device configuration, device firmware, users, groups and passwords, removing this engineering burden from system developers.

Features of Xively's cloud IoT technology :

1. Blazing fast speed and infinitely scalable connectivity, with the hardware and software to process messages from millions of devices at sub-second speeds.
2. Device management tools for provisioning, monitoring and updating devices.
3. Authentication tools for managing identities and authenticating devices.
4. Security features that offer end-to-end protection of devices and the data residing on them.

5. Remote device support that enables connected products to autonomously communicate problems to OEMs, providing context that can be used to troubleshoot devices automatically or manually.
6. Support for a device cloud that can be used to test new devices.

• Xively Python Libraries can also be used to embed python code as per the Xively APIs. A Xively Web interface is provided to be used for easy implementation of front end interface. Xively also comes with multiple language and platform support.

Q.11 How device send data to Xively ?

Ans. : • Steps for sending data from device to Xively :

1. Download the ArduinoIDE.
2. Install Xively and the HN P Client libraries.
3. Connect the hardware module to Arduino.
4. Login to Xively and register a new device.
5. Copy the API_KEY and the FEED_ID to the clipboard.
6. Download the example sketch for Arduino.
7. Change the SSID, the Password, the API_KEY and the FEED_ID in the source code.
8. Compile and upload the example.
9. Debug the program by using the Arduino Terminal.
10. Verify the data storage in Xively.
11. Switch on/off the light from Xively.

• Xively is available for many platforms since libraries for Arduino, Android, ARM mbed, C, Electric Imp, Java, JavaScript, PHP, Python and Ruby are available.

• Xively provides message bus for real time message management and routing, data services for time archiving, directory services that provides a searchable directory of objects and business services for device provisioning and management.

• Xively device support one or more channels. Each channel enables bi-directional communication between the IoT devices and Xively cloud.

Q.12 Write short note on : Thingspeak.

Ans. : • Thingspeak is an IoT analytics platform service that allows you to aggregate, visualize and analyze live data streams in the cloud. Thingspeak provides instant visualizations of data posted by your devices to Thingspeak.

• Key features :

- a) Easily configure devices to send data to Thingspeak using popular IoT protocols.
- b) Visualize your sensor data in real-time.
- c) Aggregate data on-demand from third-party sources.
- d) Use the power of MATLAB to make sense of your IoT data.
- e) Run your IoT analytics automatically based on schedules or events.
- f) Prototype and build IoT systems without setting up servers or developing web software.

• Thingspeak allows you to build an application around data collected by sensors. Features of Thingspeak include real-time data collection, data processing, visualizations, apps and plugins.

• At the heart of Thingspeak is a Thingspeak Channel. A channel is where you send your data to be stored. Each channel includes 8 fields for any type of data, 3 location fields and 1 status field. Once you have a Thingspeak Channel you can publish data to the channel, have Thingspeak process the data, and then have your application retrieve the data.

5.4 : Python Web Application Framework

Q.13 What is Django? Explain Django architecture and template system.

Ans. : • Django is a open source Python web development framework.

Django is based on the well known model-template-view architecture. It provides a unified API to a database back end.

• Django is often referred to as an MTV framework. In the MTV development pattern :

1. M stands for "Model," the data access layer. This layer contains anything and everything about the data : How to access it, how to validate it, which behaviors it has and the relationships between the data.
2. T stands for "Template," the presentation layer. This layer contains presentation-related decisions : How something should be displayed on a Web page or other type of document.
3. V stands for "View," the business logic layer. This layer contains the logic that accesses the model and defers to the appropriate template(s). You can think of it as the bridge between models and templates.

• Django was built using Python, an object-oriented applications development language which combines the power of systems languages, such as C/C++ and Java, with the ease and rapid development of scripting languages, such as Ruby and Visual Basic.

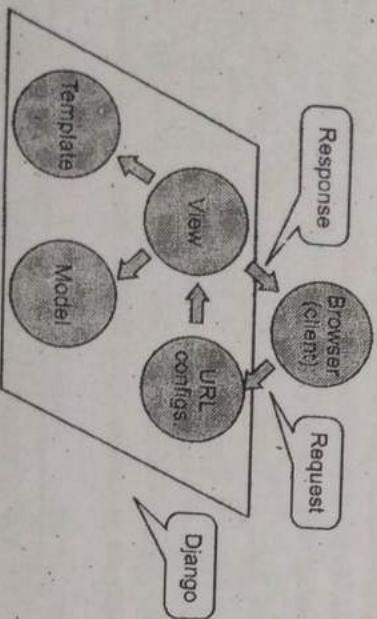


Fig. Q.13.1 Django architecture

• DJANGO MVC - MVT Pattern : The Model-View-Template (MVT) is slightly different from MVC. In fact the main difference between the two patterns is that Django itself takes care of the controller part (Software code that controls the interactions between the model and view), leaving us with the template.

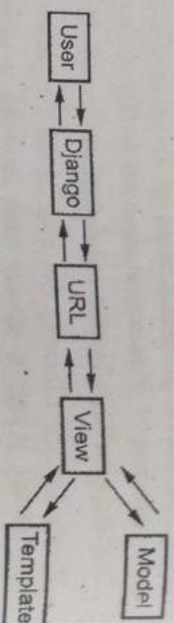


Fig. Q.13.2 MVT/MVC

- A Django template is a string of text that is intended to separate the presentation of a document from its data.
- A template defines placeholders and various bits of basic logic (i.e., template tags) that regulate how the document should be displayed.
- To use the template system in Python code, just follow these two steps :
 1. Create a template object by providing the raw template code as a string. Django also offers a way to create template objects by designating the path to a template file on the filesystem; we'll examine that in a bit.
 2. Call the render() method of the template object with a given set of variables. This returns a fully rendered template as a string, with all of the variables and block tags evaluated according to the context.

• The easiest way to create a template object is to instantiate it directly. The template class lives in the "django.template" module, and the constructor takes one argument, the raw template code.

Q.14 How Django processes a request ?

Ans. : • When an HTTP request comes in from the browser, a server-specific handler constructs the "HttpRequest" "passed to later components and handles the flow of the response processing.

• The handler then calls any available Request or View middleware. These types of middleware are useful for augmenting incoming types of requests. If either returns an "HttpResponse", proceeding bypasses the view.

• Fig. Q.14.1 shows complete flow of a Django request and response.

Model
Template

ded to separate the

of basic logic (i.e.,
be displayed.
follow these two

emplate code as a
nplate objects by
filesystem; we'll

ith a given set of
a string, with all
to the context.

ntiate it directly.
odule, and the

the browser, a
passed to later
g.

ew middleware,
nning incoming
ing for specific
se", processing

1 response.

Internet of Things

• Bugs slip by, even the best programmers, but exception middleware can help squash them. If a view function raises an exception, control passes to the Exception middleware.

• If this middleware does not return an "HttpResponse", the exception is re-raised.

• Finally, response middleware is good for post-processing an "HttpResponse" just before it's sent to the browser or doing cleanup of request-specific resources.

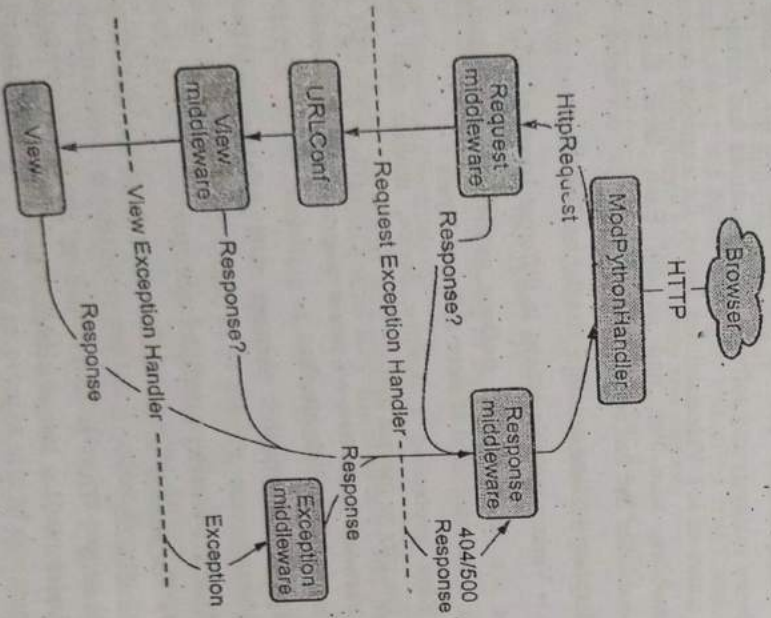


Fig. Q.14.1 Complete flow of a Django request and response

Q.15 Write a short note on Django template system.

Ans. : • A Django template is a string of text that is intended to separate the presentation of a document from its data.

• A template defines placeholders and various bits of basic logic (i.e., template tags) that regulate how the document should be displayed.

• To use the template system in Python code, just follow these two steps :

1. Create a template object by providing the raw template code as a string. Django also offers a way to create template objects by designating the path to a template file on the filesystem; we'll examine that in a bit.

2. Call the render() method of the template object with a given set of variables. This returns a fully rendered template as a string, with all of the variables and block tags evaluated according to the context.

• The easiest way to create a template object is to instantiate it directly. The template class lives in the "django.template" module, and the constructor takes one argument, the raw template code.

5.5 : Designing a RESTful Web API

Q.16 What is a REST API ?

Ans. : • REST or RESTful API design (Representational State Transfer) is designed to take advantage of existing protocols.

• While REST can be used over nearly any protocol, it usually takes advantage of HTTP when used for Web APIs.

• REST has the ability to handle multiple types of calls, return different data formats and even change structurally with the correct implementation of hypermedia.

• This freedom and flexibility inherent in REST API design allow you to build an API that meets your needs while also meeting the needs of very diverse customers.

• Unlike SOAP, REST is not constrained to XML, but instead can return XML, JSON, YAML or any other format depending on what the client requests.

- And unlike RPC, users aren't required to know procedure names or specific parameters in a specific order.
- However, there are drawbacks to REST API design. You can lose the ability to maintain state in REST, such as within sessions, and it can be more difficult for newer developers to use.
- It's also important to understand what makes a REST API RESTful, and why these constraints exist before building your API.
 - Important REST principles are as follows :
 1. **Stateless** : No client context stored on the server, each request is complete.
 2. **Cacheable** : Responses explicitly indicate their cacheability.
 3. **Layered system** : Client cannot tell if connected directly to the server (e.g. reverse proxies).
 4. **URLs** : Resources are identified using Uniform Resource Identifiers (URLs).

5.6 : IoT Security

Q.17 What are the security challenges in IoT ?

Ans. : The security challenges

- a. Devices are not reachable : Most of the time a device is not connected.
- b. Devices can be lost and stolen : Makes security difficult when the device is not connected.
- c. Devices are not crypto-engines : Strong security difficult without processing power.
- d. Devices have finite life : Credentials need to be tied to lifetime.
- e. Devices are transportable : Will cross borders.

Q.18 Explain vulnerabilities of IoT.

Ans. : • IoT devices are vulnerable largely because these devices lack the necessary built-in security to counter threats. Aside from the technical aspects, users also contribute to the device's vulnerability to threats. Here are some of the reasons these smart devices remain vulnerable :

1. Limited computational abilities and hardware limitations.
2. Heterogeneous transmission technology.
3. Components of the device are vulnerable.
4. Users lacking security awareness. Lack of user security awareness could expose smart devices to vulnerabilities and attack openings.
- An IoT device can have one or multiple vulnerabilities that make it an easy target for hackers to gain access to a network and move laterally to more critical devices or systems.
- IoT vulnerabilities due to participation of the number of layers, hardware sublayers and software in applications and services.
- The nature of IoT Vulnerabilities varies, for example, sensors, machines, automobiles, wearables, and so on. Each faces different kind of vulnerabilities and has complex security and privacy issues.

Q.19 Explain various possible attacks on IoT layers.

Ans. : • Various possible attacks in different layers of IoT.

- There are various attack surfaces available for attackers, protection needs to be considered at three different layers :
 - 1) **Edge protection** : Ensures device, mobile app and web app integrity to prevent devices from becoming attack entry points.
 - 2) **Network protection** : Secures communication channels to prevent man-in-the-middle attacks.
 - 3) **Cloud protection** : Assures data privacy and prevents data leakage.

Q.20 What are security principles of IoT ?

Ans. : • The security of the Internet of Things, the following principles can be established.

- a) **Identity** : Trust is always tied to an identity. Therefore every device needs a unique identity that can't be changed. The device must also be able to prove its identity at all times.
- b) **Positive intention** : The device and linked service have positive intentions.

ity awareness
ick openings.
hat make it an
move laterally

er of layers,
es,
ors, machines,
rent kind of

s, protection

nd web app
v points.

is to prevent

vents data

g principles

efore every
The device

ave positive

- c) **Predictability and transparency :** The functional scope of the service provided by devices is known to its full extent. There are no undocumented (secret) functions. The behaviour of the system can be checked at any time by independent third parties.
- d) **Reputation :** An increasing number of positive interactions between the things gradually form a reputation based intelligent network.

Q.21 What are the key requirement for any IoT security solution ?

Ans. : • The key requirements for any IoT security solution are :

1. Device and data security, including authentication of devices and confidentiality and integrity of data.
 2. Implementing and running security operations at IoT scale.
 3. Meeting compliance requirements and requests.
 4. Meeting performance requirements as per the use case.
- Application layer : Verification and user's confidentiality
 - Support layer : Various encryption algorithms
 - Network layer : Distributed denial of service attack
 - Physical layer : Authentication.

5.7 : Threat Modelling and Key Elements of IoT Security

Q.22 Describe threat modelling of IoT security.

Ans. : • Threat modeling is an approach for analyzing the security of an application. It is a structured analysis to identify, quantify and address the information security risks associated with an application or a system.

- IoT devices face many threats, including malicious data that can be sent over authenticated connections, exploiting vulnerabilities. Such attacks frequently exploit many weaknesses, including but not limited to :
 - a) Failure to use code signature verification and secure boot and
 - b) Poorly implemented verification models which can be bypassed.
- Attackers often use those weaknesses to install backdoors, sniffers, data collection software, file transfer capabilities to extract sensitive

- information from the system, and sometimes even Command & Control (C&C) infrastructure to manipulate system behavior.
- It is well known that IoT devices are generally lagging in terms of network and information security. This is either due to :
 - a) Lax manufacturing standards
 - b) Devices that do not have the computational horsepower or storage space to be secured.

• Even if one device is properly secured, unsecured devices can still exist in the organization's ecosystem. This entirely bypasses the scope and reach of IT security teams and opening up entire networks to data breaches. Architecturally-based IoT threat modeling can reveal these IT security bypasses.

• Without specific IoT security, an organization can easily lose control of their IoT ecosystem attack surface. Organizations can, for example, dictate and enforce policies that no personal smart devices are brought into the work environment.

• All that care and monitoring can be thrown out the window if one employee logs into the IT system while working from home. While it is possible to prevent employees from bringing smart devices into the work environment, it would be difficult to manage visitors with IoT devices from entering the building. Securing the IoT ecosystem simply cannot be done by policy mandates.

• Threat modeling is a procedure for optimizing network security by identifying objectives and vulnerabilities, and then defining countermeasures to prevent, or mitigate the effects of, threats to the system.

• Threat modeling is a systematic approach for developing resilient software. It identifies the security objective of the software, threats to it, and vulnerabilities in the application being developed. It will also provide insight into an attacker's perspective by looking into some of the entry and exit points that attackers are looking for in order to exploit the software.

Vulnerability List	Identified Threats
Injection	XML Schema Poisoning Accessing/Intercepting/Modifying HTTP Cookies HTTP Response Splitting
Broken Authentication and Session Management	JSON Hijacking (aka JavaScript Hijacking) Read Sensitive Strings Within an Executable
Cross Site Scripting (XSS)	Cross Site Scripting through LOG Files Cross Site Scripting in Error Pages
Insecure Direct Object Reference	Privilege Abuse Relative Path Traversal
Security Misconfiguration	Padding Oracle Crypto Attack Target Programs with Elevated Privileges
Sensitive Data Exposure	Interception Encryption Brute Forcing
Missing Function Level Access Control	Cross Zone Scripting Directory Indexing
Cross Site Request Forgery (CSRF)	Cross Site Request Forgery
Using Components with known Vulnerabilities	Using commons-collections for Java Deserialization Vulnerability
Unvalidated Redirects and Forwards	Fake the Source of Data

Q.23 Explain identity establishment issue with respect to IoT security.

Ans. : • The identity management in IoT is performed by interchanging finding information between the things for first time connection.
• This process is affected to overhear which can lead to man-in-the-middle attack and thus can threaten the whole IoT structure.
• Hence, there needs to be some pre-defined identity management entity or hub which can observe the relation process of devices by applying cryptography and other techniques to prevent identity theft.
• IoT requires security computes at all three layers; at physical layer for data collection, at network layer for overpower and dispatch and at application layer to maintain confidentiality, authentication and integrity.

Q.24 Explain key elements of IoT security.

Ans. : • Fig. Q.24.1 shows key elements of IoT security.

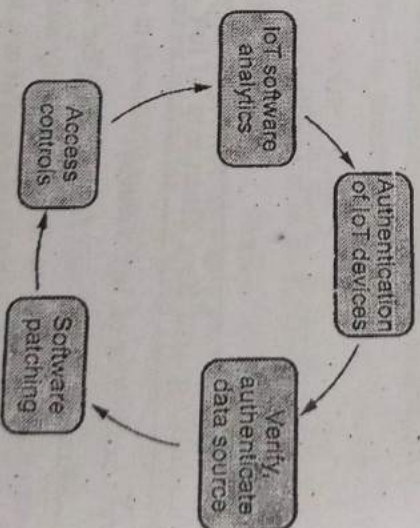


Fig. Q.24.1 Elements of IoT security

• Authenticate IoT devices before they integrate with the centralized network. This step avoids the risk of spoofing attacks on the IoT device which appears as a legitimate device on your network. Attacked devices collect data from other networked device or transmit malicious data to remaining devices on the network.

- It is essential to authenticate and verify software source running on your IoT device. Software not authenticated faces the risk of being compromised and the device cannot detect such tampering issues unless software is supported with a digital signature of the vendor.
- Software patching avoids the IoT device from being compromised. However, software updates from authenticated sources must be accepted only. Software patches minimize risks of data loss or interference with business operations. For instance, a device is put for updating; all local data is written on a central network, other devices know when the updating device goes offline and the update is performed, verified before returning back to the operational mode.
- Access controls secure IoT devices as well as the enterprise. Users are assigned roles to perform specific operations. Roles include querying current state of the IoT devices, software updates on these devices and change device configurations. Other systems present on the network work on the principle of least privilege that allows users only a minimal set of privileges to perform business functions and technical processes. This method restricts damage caused due to a security breach of user's personal data.
- IoT software analytics must be designed on the basis of anomaly detection. Basic guidelines may be pre-defined, and variation from these guidelines should alert the user. For instance, higher-than-expected traffic from some devices would alert users that the devices have been compromised and are in use even after a malicious attack. Considering the response to such anomalous behavior, you can perhaps switch off the affected devices or remove them from the connected network.

5.8 : Security Model for IoT

Q.25 What are challenges encounter while designing IoT ?

Ans. : • IoT enables a constant transfer and sharing of data among things and users. In such a sharing environment, authentication, authorization, access control and non-repudiation are important to ensure secure communication.

- The high level of heterogeneity, coupled to the wide scale of IoT systems, is expected to magnify security threats of the current Internet. The high number of inter-connected devices arises scalability issues.
- IoT systems integrate in a seamless way physical objects, data and computing devices into a global network of information about "smart things".

Fig. Q.25.1 shows high level security challenges of IoT.

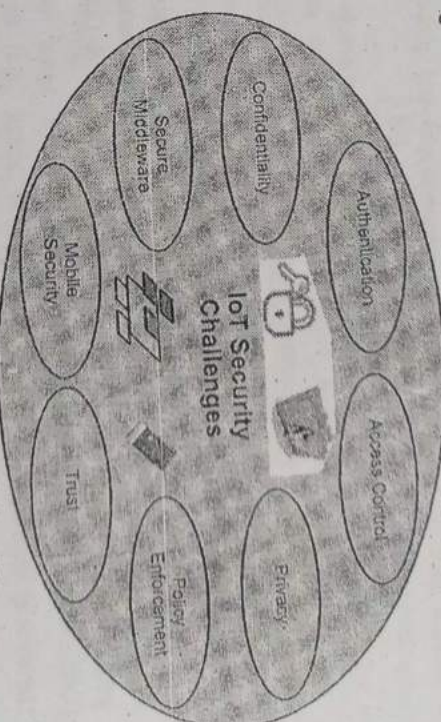


Fig. Q.25.1 IoT security challenges

- Access control refers to the permissions in the usage of resources, assigned to different actors of a wide IoT network.
- System safety is highly application - or application domain-specific. Trust Model that provides data integrity and confidentiality, and endpoint authentication and non-repudiation between any two system-entities that interact with each other.
- The trust requirements in IoT are related to identity management and access control issues. The IoT-A Privacy Model depends on the following functional components : Identity Management, authentication, authorization, trust and reputation.

- Communication security : IoT systems are heterogeneous. Not only because of the variety of the entities involved, but also because they include devices with various capabilities in terms of communication and processing.
- Communication security model must not only consider the heterogeneity of the system, but it also should guarantee a balance between security features, bandwidth, power supply and processing capabilities.
- IoT devices face many threats, including malicious data that can be sent over authenticated connections, exploiting vulnerabilities. Such attacks frequently exploit many weaknesses, including but not limited to :
 - a) Failure to use code signature verification and secure boot and
 - b) Poorly implemented verification models which can be bypassed.
- Attackers often use those weaknesses to install backdoors, sniffers, data collection software, file transfer capabilities to extract sensitive information from the system, and sometimes even Command & Control (C&C) infrastructure to manipulate system behavior.
- The security challenges are as follows :
 - a. Devices are not reachable : Most of the time a device is not connected.
 - b. Devices can be lost and stolen : Makes security difficult when the device is not connected.
 - c. Devices are not crypto-engines : Strong security difficult without processing power.
 - d. Devices have finite life : Credentials need to be tied to lifetime.
 - e. Devices are transportable : Will cross borders.
- IoT system has a cloud database that is connected to all your devices. These devices are connected to the internet and it could be accessed by the cybercriminals and hackers. As the number of connected devices increases, chances for hackers to breach the security system gets increased.

Q.26 Explain security model for IoT.

Ans : Fig. Q.26.1 shows security model for IoT.

Application Layer	IoT Application	
	Application support layer (Middleware technology security)	
Transportation Layer	Local Area Network	
	Core Network (Internet Security)	
Perception Layer	Access Network (Ad hoc security, GPS security, WiFi security)	
	Perception Network (RFID security) (WSN Security)	
	Perception Node (RSN Security)	

Fig. Q.26.1 Security model for IoT

- Data and privacy protection is one of the application challenges of IoT. In IoT, RFID systems, WSNs sensors perceive for the end of the information technology, which protect the integrity and confidentiality of information by the password encryption technology.
- There are many ways to encrypt data and information, such as random hash lock protocol (hash function), hash chain protocol, extract key from an infinite channel, encrypted identifier and so on.
- Identity authentication and access control can determine the identity, prevent disguised attacks to ensure the authenticity, validity of the information and so on.
- IoT must ensure the security of all layers. In addition, IoT security should also include the security of whole system crossing the perception layer, transportation layer and application layer.
- Perception layer includes RFID security, WSNs security, RSN security and any others.

- Transportation layer includes access network security, core network security and local network security. There are 3G access network security, Ad-hoc network security, WiFi security and so on for these sublayers.
- Perception layer is mainly about information collection, object perception and object control. Perception layer can be divided into two parts : Perception node (sensors or controllers, etc.), perception network that communicates with transportation network.
- Transportation layer mainly provides ubiquitous access environment for perception layer, perception of information transmission and storage and application layer load other related businesses. Transportation layer can be divided into three layers by function : The access network, the core network and local area. It is a combination of a variety of heterogeneous networks.
- The application support layer, an advanced layer above the transportation layer, supports all sorts of business services and realizes intelligent computation and resources allocation in screening, selecting, producing and processing data. During the whole process, the application support layer can recognize valid data, spam data and even malicious data and filter them in time.

END... 